**МИНОБРНАУКИ РОССИИ**
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**
**Кафедра МОЭВМ**

# ОТЧЕТ

**по лабораторной работе №1**
**по дисциплине «Объектно-ориентированное проргаммирование»**
**Тема: Реализация игры с пошаговым перемещением игрока и сражением с врагами.**

Студент гр. 4381 _____ Захаренко А.В.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2025

**Цель работы.**

Начать реализовывать игру с пошаговым перемещением игрока и сражением с врагами.

**Задание.**

1. Создать класс игрока, который должен хранить информацию об игроке (его жизни, урон, очки, и т.д. - студент сам определяет необходимые для работы характеристики). Объект класса игрока должен перемещаться по карте. Если у игрока кончаются жизни, то происходит конец игры.

2. Создать класс врага, который хранит параметры жизней и урона. Объектами класса врага управляет компьютер. При перемещении, если враг пытается перейти на клетку с игроком, то перемещение не происходит, и игроку наносится урон.

3. Создать класс квадратного/прямоугольного игрового поля, по которому перемещаются игрок и враги. Игровое поле не должно быть меньше 10 на 10 клеток, и не больше 25 на 25 клеток. Размеры поля задаются через конструктор. Рекомендуется для хранения информации об отдельных клетках поля создать отдельный класс.

Реализовать конструкторы перемещения и копирования для поля, а также соответствующие операторы присваивания с копированием и перемещением (должна происходить глубокая копия).

4. Реализовать непроходимые клетки на поле. При попытке врагов или игрока перейти на такую клетку, перемещение не происходит. Заполнение поля непроходимыми клетками происходит в момент создания поля.

5. Добавить возможность для игрока переключаться на ближний или дальний бой с изменением значения наносимого урона. Такое переключение требует один ход.

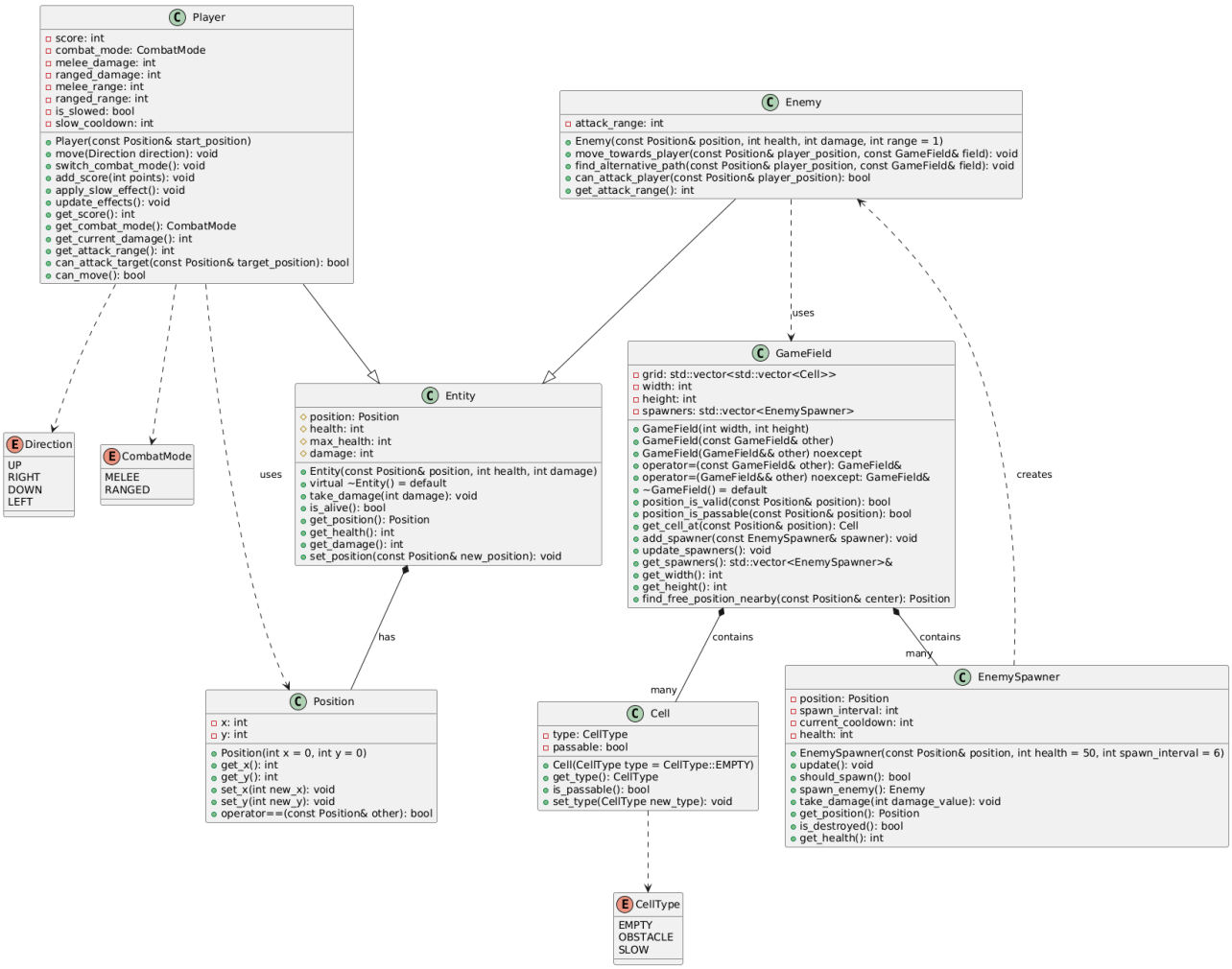
6. Добавить класс вражеского здания. Такое здание размещается на карте, и раз в несколько ходов создает нового врага возле себя. Количество ходов до создания нового врага задается в конструкторе.

7. Реализовать замедляющие клетки на поле. Если игрок переходит на такую клетку, то он он не может двигаться на следующий ход.

**Примечания:**

- Не забывайте для полей и методов определять модификаторы доступа
- Для обозначения переменной, которая принимает небольшое ограниченное количество значений, используйте enum
- Не используйте глобальные переменные
- При реализации перемещения, не должно быть лишнего копирования
- У поля не должно быть методов возвращающих указатель на поле в явном виде, так как это небезопасно
- Информация о координатах игрока не должна дублироваться у игрока и у поля

# UML-диаграмма классов.



**Выполнение работы.**

Расспишем работу программы: реализацию класов, их назначение и основные архитектурные решения.

1) Класс Position — нужен для определения координат предметов на поле.

Поля класса:

int x, y — целые координаты позиции на карте (карта предоставляет из себя ПДСК с осями x вправо, y вверх)

Методы класса:

int get_x() const — геттер для x

int get_y() const — геттер для y

void set_x(int new_x) — сеттер для x

void set_y(int new_y) — сеттер для y

Также для удобства сравнения двух позиций был перегружен опеартор сравнения:

bool operator==(const Position& other) const

2) Класс Entity — базовый класс сущностей на поле: игрок, враги и возможнжные будущие союзники игрока. Было принято решение добавить этот класс во избежание дублирования кода при описании этих сущностей, так как они обладают общими свойствами (полями) и действиями (методами).

Поля класса:

Position position — позиция сущности

int health — текущий запас здоровья

int max_health — максимальный (начальный) запас здоровья

int damage — наносимый урон

Методы класса:

explicit Entity(const Position& position, int health, int damage) — конструктор

virtual ~Entity() = default — виртуальный деструктор; был добавлен для гарантии вызова деструкторов производных классов.

void take_damage(int damage) — получить урон

bool is_alive() const — жива ли сущность

Position get_position() const — геттер для получения позиции сущности

int get_health() const — геттер для получения текущего здоровья сущности

int get_damage() const — геттер для получения урона, наносимого сущностью

void set_position(const Position& new_position) — сеттер для задания позиции

3) Класс Player — класс игрока, за которого игрет пользователь. Является наследником класса Entity. Никаких методов не переопределяет, только имеет дополнительные уникальные атрибуты.

Поля класса:

int score — текущее количество очков игрока

CombatMode combat_mode — режим атаки (рукопашный или дальний)

int melee_damage — урон от рукопашного боя

int ranged_damage — урон от дальнего боя

int melee_range — дальность ближнего боя

int ranged_range — дальность дальнего боя

bool is_slowed — замедлен ли игрок (наступил ли он на замедляющую клетку)

int slow_cooldown — длительность замедления игрока после перемещения на замедляющую клетку

Методы класса:

explicit Player(const Position& start_position) — конструктор

void move(Direction direction) — перемещение по заданному направлению (вверх/ вниз/ вправо/ влево)

void switch_combat_mode() - сменить режим боя

void add_score(int points) — накопить очки

void apply_slow_effect() - применить на себя замедляющий эффект (пропуск хода)

void update_effects() — обновить эффекты, наложенные на игрока

int get_score() const — геттер для получения текущих очков

CombatMode get_combat_mode() const — геттер для получения режима боя

int get_current_damage() const — геттер для получения урона, наносимого игроком

int get_attack_range() const — геттер для получения дальности атаки

bool can_attack_target(const Position& target_position) const — может ли игрок атаковать цель на указанной позиции

bool can_move() const — может ли игрок перемещаться (не замедлен ли он)

4) Класс Enemy — класс врага, за которого играет компьютер по определенным правилам. То есть перемещения и атаки врага должны быть автоматизированными. Данный класс так же является насследником класса Entity. Никаких методов не переопределяет.

Поля класса:

int attack_range — дальность атаки, по умолчанию 1

Методы класса:

Enemy(const Position& position, int health, int damage, int range = 1) — конструктор

void move_towards_player(const Position& player_position, const GameField& field) — движение по направлению к игроку, посредством стремления уменьшить разницу между координатами.

void find_alternative_path(const Position& player_position, const GameField& field) — поиск альтернативного пути. Вспомогательный к предыдущему; вызывается если в основном маршруте следующая клетка это препятствие, поэтому надо это препятствие обходить.

bool can_attack_player(const Position& player_position) const — может ли враг нанести урон игроку (то есть стоит ли враг в 1 клетке (не по диагонали) от игрока)

int get_attack_range() const — геттер дальности атаки

5) Класс Cell — класс одной клетки на поле. Нужен для того, чтобы быть главной составляющей частью поля, так как поле в данной реализации это вектор векторов из клеток (двумерный массив клеток).

Поля класса:

CellType type — тип клетки (пустая, препятствие или замедляющая)

bool passable — параметр, отвечающий за то, проходиа ли клетка.

Методы класса:

explicit Cell(CellType type = CellType::EMPTY) — конструктор

CellType get_type() const — геттер для получения типа клетки

bool is_passable() const — является ли клетка проходимой (не препятсвие ли)

void set_type(CellType new_type) — сеттер для установки типа клетки

6) Класс EnemySpawner — класс вражеского здания, откуда появляются враги с интервалом в определенное количество ходов игрока.

Поля класса:

Position position — позиция спавнера на поле

int spawn_interval — интервал появления новых врагов из здания

int current_cooldown — значение отсчета до появления нового врага (перезарядка)

int health — текущий запас здоровья здания.

Методы класса:

explicit EnemySpawner(const Position& position, int health = 50, int spawn_interval = 6) — конструктор

void update() — уменьшить время до спавна нового врага на 1

bool should_spawn() const — настало ли время для спавна нового врага

Enemy spawn_enemy() - заспавнить нового врага

void take_damage(int damage_value) — получить урон

Position get_position() const — геттер для получения позиции здания

bool is_destroyed() const — уничтожена ли башня

int get_health() const — геттер для получения запаса здоровья

7) Класс Field — класс игрового поля с опеределенными параметрами длины и высоты, по которому перемещаются сущности. Во время создания поля на нем случайным образом генерируются особые клетки: непроходимые для всех сущностей клетки (препятствия) и замедляющие игрока клетки. Также на поле размещаются игрок, враги и вражеское здание (будут в основном цикле игры).

Поля класса:

std::vector<std::vector<Cell>> grid — сетка поля

int width — длина поля

int height — высота поля

std::vector<EnemySpawner> spawners — динамический массив спавнеров, расположенных на поле.

Методы класса:

Приватные:

void initialize_grid() - инициализировать сетку (заполнить ее пустыми клетками)

void generate_special_cells() - сгенерировать и добавить на поле особые клетки (препятствия и замораживающие клетки) по 10% каждых

Публичные:

GameField(int width, int height) — конструктор, внутри которого корректируются значения длины и высоты, если они больше/меньше граничных 25/10. Также в конструкторе происходит инициализация сетки и добавление туда особых клеток.

GameField(const GameField& other) — конструктор копирования

GameField(GameField&& other) noexcept — конструктор перемещения

GameField& operator=(const GameField& other) — оператор присваивания с копированием

GameField& operator=(GameField&& other) noexcept — оператор присваивания с перемещением

~GameField() = default — деструктор (в данному случае по умолчанию)

bool position_is_valid(const Position& position) const — является ли позиция корректной в плане выхода за границы

bool position_is_passable(const Position& position) const — является ли позиция проходимой (не барьер ли она)

Cell get_cell_at(const Position& position) const — получить клетку по заданной позиции

void add_spawner(const EnemySpawner& spawner) — добавить спавнер на поле

void update_spawners() — обновить отсчет до нового спавна для каждого спавнера на поле

std::vector<EnemySpawner>& get_spawners() - получить вектор из спавнеров.

int get_width() const — геттер для получения длины

int get_height() const — геттер для получения высоты

**Тестирование.**

Было проведено тестирование разработанной части игры посредством демонстрации выполнения основного функционала.

Результат отладочного вывода в консоль приведен в приложении Б.

**Вывод.**

Во время выполнения лабораторной работы было положено начало разработки игры. Была разработана начальная объектно-ориентированная архитектура проекта: классы игрока, врага, вражеского здания и игрового поля. А также продемонстрирован базовый функциаонал реализованных классов.

## Приложение Б.

Результат отладочного вывода в консоль:

```
*************** ДЕМОНСТРАЦИЯ ОСНОВНОГО ФУНКЦИОНАЛА ***************

--- Создание игрового поля 12x15 ---
инициализация поля
препятствие на позиции (7, 3)
препятствие на позиции (11, 3)
препятствие на позиции (0, 8)
препятствие на позиции (6, 8)
препятствие на позиции (4, 9)
препятствие на позиции (4, 10)
препятствие на позиции (8, 11)
препятствие на позиции (10, 11)
препятствие на позиции (8, 12)
препятствие на позиции (1, 13)
препятствие на позиции (4, 13)
препятствие на позиции (10, 13)
препятствие на позиции (6, 14)
препятствие на позиции (10, 14)

Размер поля: 12x15

--- Создание игрока ---
Игрок создан в позиции: (3, 3)
Здоровье игрока: 100
Начальный урон: 15

--- Создание врагов ---
Враг 1 создан в позиции: (5, 5)
Враг 2 создан в позиции: (8, 8)

--- Демонстрация перемещения игрока ---
Начальная позиция игрока: (3, 3)
Движение ВПРАВО -> позиция: (4, 3)
Движение ВВЕРХ -> позиция: (4, 4)

--- Демонстрация режимов боя ---
Текущий режим: БЛИЖНИЙ
Текущий урон: 15
Переключение режима -> ДАЛЬНИЙ
Новый урон: 8

--- Проверка валидности и проходимости клеток поля ---
Позиция (2,2) валидна: 1
Позиция (2,2) проходима: 1
Позиция (25,25) валидна: 0

--- Проверка замедляющих клеток ---
Может ли игрок перемещаться сейчас: 1
*Игрок попал в замедляющую клетку*
Может ли игрок перемещаться сейчас: 0
*По прошествии времени замедления*
Может ли игрок перемещаться сейчас: 1

--- Демонстрация получения урона ---
```

```
Здоровье врага 1 до атаки: 50
Здоровье врага 1 после получения 20 урона: 30
Враг 1 жив: 1
Здоровье врага 1 после получения 40 урона: 0
Враг 1 жив: 0

--- Демонстрация ходов врага по направлению к игроку ---
Позиция врага 2 до движения: (8, 8)
Позиция игрока: (4, 4)
Позиция врага 2 после 1-го движения к игроку: (8, 7)
Позиция врага 2 после 2-го движения к игроку: (7, 7)
Позиция врага 2 после 3-го движения к игроку: (7, 6)
Позиция врага 2 после 4-го движения к игроку: (6, 6)

--- Демонстрация спавнера врагов ---
Спавнер создан в позиции: (10, 10)
Интервал спавна: 3 хода
Ход 1: Ожидание...
Ход 2: Ожидание...
Ход 3: Создан новый враг!
Ход 4: Ожидание...
Ход 5: Ожидание...
Ход 6: Создан новый враг!

Здоровье спавнера: 40
Здоровье спавнера после получения 30 ед урона: 10
Спавнер разрушен? 0
Здоровье спавнера после получения еще 30 ед урона: 0
Спавнер разрушен? 1

--- Проверка игровых состояний ---
Игрок жив: 1
Враг 1 жив: 0
Враг 2 жив: 1

--- Проверка возможности атаки игроком ---
Текущая позиция игрока: (4,4)
Режим: БЛИЖНИЙ (дальность 1)
Можно атаковать близкую цель (5,4): 1
Можно атаковать дальнюю цель (6,6): 0
Режим: ДАЛЬНИЙ (дальность 2)
Можно атаковать близкую цель (5,4): 1
Можно атаковать дальнюю цель (6,6): 1

--- Проверка возможности атаки врагом ---
Может ли враг на позиции (3,3) атаковать игрока на позиции (4,4): 0
Может ли враг на позиции (4,3) атаковать игрока на позиции (4,4): 1
```